

Curso Inicial Front-End

Clase 20: El Ecosistema React: Node, NPM y Proyectos

Módulo 3: Desarrollo con React.js y Ecosistema Moderno



Presentación

¡Bienvenido al estándar de la industria! En esta clase daremos el salto a **React**, la librería que cambió para siempre el desarrollo frontend. Aprenderemos a pensar en **Componentes** y configuraremos nuestro primer entorno profesional con **Vite**. Entenderás por qué **React** es tan rápido gracias al **Virtual DOM** y cómo organizar tus proyectos siguiendo los principios de arquitectura moderna.



Objetivos

- Comprender el rol de Node.js y NPM en el desarrollo frontend moderno.
- Configurar y ejecutar un proyecto **React** desde cero usando npm init **Vite**@latest.
- Comprender el concepto de Single Page Application (SPA) y la diferencia técnica entre el DOM Real y el **Virtual DOM**.
- Aplicar el principio **SOLID (S)** para decidir cuándo y cómo crear componentes reutilizables.



Bloques Temáticos

- Tema 1: Introducción a bibliotecas de UI y Single Page Applications (SPA).
 - El Cimiento
 - Bibliotecas de UI y SPA
 - Puesta en marcha
 - El Corazón del Proyecto
 - Introducción a Componentes
 - Estructura y Organización
-

Tema 1: Introducción a bibliotecas de UI y Single Page Applications (SPA).

1. El Cimiento

Antes de escribir una sola línea de código, necesitamos conocer las herramientas que hacen posible el desarrollo moderno.

¿Qué es Node.js?

Históricamente, **JavaScript** solo vivía dentro del navegador. **Node.js** es un entorno de ejecución (runtime) que permite ejecutar **JavaScript** en la computadora (fuera del navegador). Este internamente posee el mismo motor que ejecuta **JavaScript** en el navegador llamado **V8**.

- **En el Frontend:** No usamos **Node.js** para que corra nuestra aplicación en el navegador de los usuarios, sino para ejecutar herramientas de desarrollo (compiladores, servidores locales de prueba, empaquetadores).

¿Qué es NPM (Node Package Manager)?

Es el **gestor de paquetes** por defecto de Node.js. Es la librería/tienda de software más grande del mundo.

- **Su función:** Nos permite **descargar e instalar herramientas y librerías** creadas por otros desarrolladores (como **React** , Vite, librerías de estilos, etc.) con comandos simples en la terminal.

2. Bibliotecas de UI y SPA

Una biblioteca de **programación** es como una **caja de herramientas** con piezas **prefabricadas**. En lugar de tener que construir cada pequeña parte de tu aplicación desde cero (*como botones, menús o cálculos complejos*), usas estas piezas que otros programadores ya **crearon, probaron y optimizaron** para que tú solo tengas que **enfocarte en armar tu proyecto de forma mucho más rápida y eficiente**. Tú decides cuándo y cómo usar cada herramienta, manteniendo siempre el control total sobre el resultado final.

En ese sentido a lo largo de esta cursada usaremos **React** que es una biblioteca orientada a la **creación de interfaces de usuario**. Conocidas también como **UI** son el "**puente**" visual y funcional que traduce las complejas instrucciones de una máquina en algo que nosotros, como humanos, podemos entender, tocar y navegar con facilidad.

React permite crear aplicaciones **SPA (Single Page Applications)**. A diferencia del **HTML** antiguo donde cada **clic** recargaba toda la página, **React** permite actualizar de manera eficiente únicamente los componentes específicos que cambian. Esto resulta en una experiencia de navegación fluida y rápida, similar a la de una aplicación nativa móvil.

I Comparativa: Virtual DOM vs. DOM Real

- **DOM Real (Vanilla JS):** Es lento. Si cambias un dato, el navegador debe recalcular el diseño de toda la página.

- **Virtual DOM (React):** Es un "clon" ligero en memoria. React detecta el cambio en el clon, lo compara con la realidad y **solo aplica el parche** en el elemento exacto. ¡Es la clave de su velocidad!

En lenguaje simple: El Virtual DOM es como una libreta de bocetos. React dibuja ahí primero, ve qué cambió realmente, y solo entonces va a la pared real a pintar el cambio. Es mucho más rápido que pintar toda la pared cada vez.

Traducción Técnica: El **Virtual DOM** es una representación ligera del DOM real en forma de objetos de JavaScript. React utiliza un algoritmo de **Reconciliación (Diffing)** para comparar el árbol virtual nuevo con el anterior y generar un conjunto mínimo de operaciones de manipulación del DOM real para sincronizarlos.

2. Puesta en Marcha

Para poner en marcha nuestro primer proyecto con React usaremos Vite que es la herramienta industrial para crear proyectos. Es **10x** veces más rápido que las herramientas viejas.

Guía paso a paso para inicializar tu proyecto:

1. **Abrir Terminal:** Ve a tu carpeta de proyectos.
2. **Comando de creación:** `npm init vite@latest`
3. **Project name:** Ingresa el nombre de tu carpeta (ej. `mi-proyecto`).
4. **Selección de Framework:** Elige React con las flechas.
5. **Selección de Variante:** Elige JavaScript.
6. **Instalación:** Entra a la carpeta (`cd mi-proyecto`) y ejecuta `npm install` para bajar las herramientas.
7. **Ejecución:** Escribe `npm run dev` y abre el link que te da la consola. Esto levantará un **servidor** local rápido. Haz **Ctrl + Clic** en la URL que te

muestre la terminal (normalmente **http://localhost:5173**) para ver tu App en vivo. Si debes detener el servidor presiona **Ctrl + C**.

Modo Directo y Rápido (Recomendado) 💡

*Si quieres saltarte las preguntas y crear el proyecto inmediatamente con la plantilla de **React** ya configurada, puedes pasar parámetros directamente:*

```
npm init vite@latest mi-primer-proyecto -- --template react
```

Nota técnica: El doble guión -- es necesario en npm para indicarle que todos los parámetros siguientes (--template react) deben ser pasados directamente al comando ejecutable interno de vite, y no a npm mismo.

| El Scaffolding (Andamiaje) del Proyecto

Al crear el proyecto, verás estos archivos clave:

- **index.html**: La raíz. Tiene un `<div id="root"></div>` vacío donde **React** "atterrizará" toda tu App.
- **src/main.jsx**: El puente. Es el archivo más importante porque conecta tu código de **React** con el `div#root` del **HTML** real.
- **src/App.jsx**: Tu primer componente. Aquí empieza la magia.
- **.jsx (Extensión)**: Significa "**JavaScript** XML". Permite escribir **HTML** directamente dentro de funciones **JS**. Es obligatorio para que **Vite** entienda que hay diseño visual.

En lenguaje simple: JSX es una forma de escribir cómo quieres que se vea tu página usando etiquetas que parecen **HTML**, pero dentro de tu archivo de **JavaScript**.

Traducción Técnica: JSX es una extensión sintáctica de **JavaScript** (ECMAScript). No es interpretada directamente por el navegador, sino que un transpilador (como Babel o el motor de **Vite**) la convierte en llamadas a funciones `React.createElement()` que generan objetos del Virtual DOM.

3. El Corazón del Proyecto

El archivo `package.json` es el manual de instrucciones de tu proyecto.

- **Scripts:** Define comandos como `dev` (desarrollo) o `build` (para subir a internet).
- **Dependencies:** Lista todas las librerías que usa tu proyecto.

Módulos: ESM Modules vs. CommonJS

- **CommonJS (Viejo):** Usaba `require()`. Era pesado y lento.
- **ESModules (Moderno):** Usa `import` y `export`. Es el estándar de **Vite**.
- **Configuración:** En el `package.json`, verás `"type": "module"`. Esto le dice a **Node.js** que use el sistema moderno de importaciones.

4. Introducción a Componentes

Un componente es una pieza de LEGO. Pero, ¿cuándo debemos crear uno?

Principio SOLID: Responsabilidad Única (S)

Un componente debe hacer **una sola cosa bien**.

- **¿Cuándo componetizar?:** Si el código se repite en otro lado, o si tu archivo supera las 100-150 líneas, es hora de extraer piezas (ej: un `Navbar`, un `Footer`, un `CardProducto`).

En lenguaje simple: La Responsabilidad Única significa que cada pieza de tu código debe tener un solo trabajo. Si un botón también intenta validar un formulario y enviar un mail, es mejor dividirlo en piezas más pequeñas.

Traducción Técnica: El **Principio de Responsabilidad Única (SRP)** de SOLID establece que un módulo o clase debe tener una sola razón para cambiar. En **React**, esto se traduce en componentes atómicos que gestionan una única lógica de representación o comportamiento, facilitando el testeo y la mantenibilidad.

Componentes de Clase vs. Funcionales

- **De Clase (Antiguo):** Usaban `class` y `this`. Son difíciles de leer y más lentos.
- **Funcionales (Actual):** Son simples funciones de **JS** que usan **Hooks**. Es lo que aprenderás aquí porque es el estándar moderno de la industria.

5. Estructura y Organización

Para no perderte en el caos, organiza tu carpeta `src` así:

```
src/  
├── assets/      # Imágenes, logos, estilos globales.  
├── components/  # Piezas pequeñas y reutilizables (Botones, Inputs).  
├── pages/       # Vistas completas de la App (Home, Login, Perfil).  
├── App.jsx      # Componente raíz.  
└── main.jsx     # Punto de entrada.
```

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **Node.js:** <https://nodejs.org/es/docs/>
- **React:** <https://es.react.dev/>
- **Vite:** <https://vitejs.dev/guide/>